



Contents

1. **Single Layer Neural Networks.** We look at how a single layer network performs on a toy dataset
2. **Multilayer Perceptrons.** We look at how multiple layers perform on toy datasets
3. **Handwritten digit recognition in sklearn.** Learning to classify numbers using the libraries in sklearn
4. **Handwritten digit recognition in Pytorch.** Learning to classify numbers using libraries in Pytorch

Finally, try working through some tutorials in Pytorch and play around with the different hyperparameter choices. Links to tutorials are at the end of the notebook.

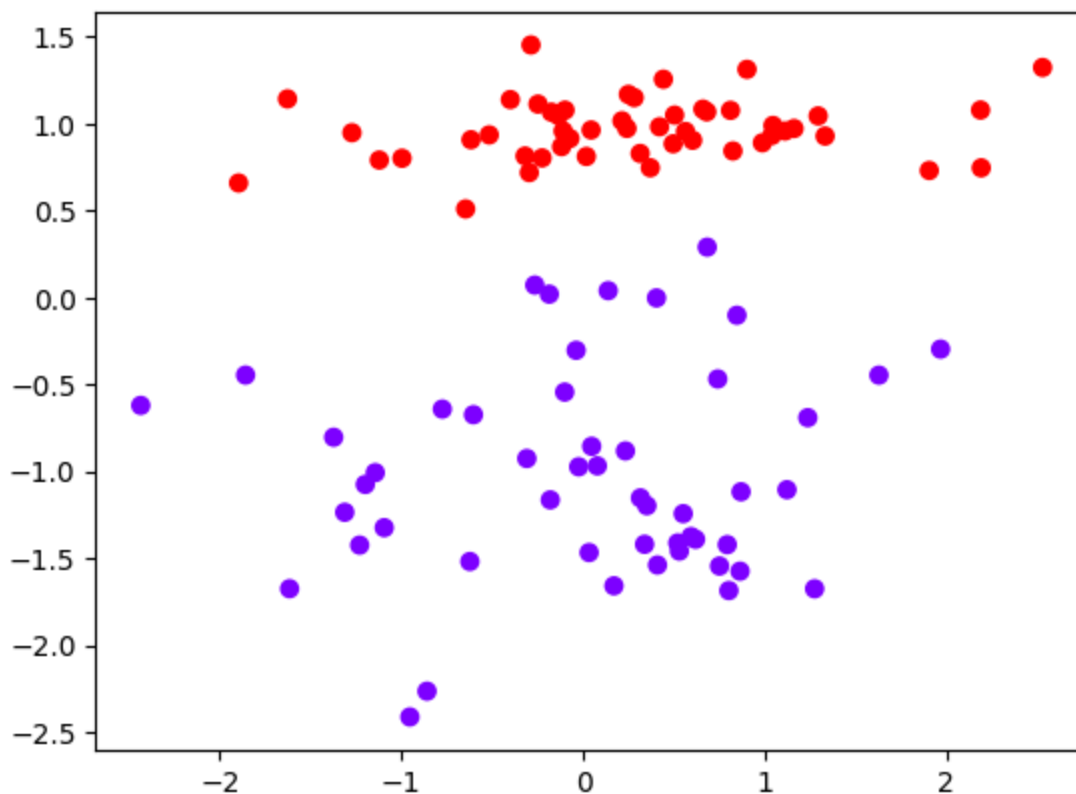
Single Layer Neural Network

In this question we apply a single-layer neural network to a linearly separable toy data set.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_moons, make_circles, make_classification
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron
```

We start by generating and visualizing some data

```
In [ ]: X1, y1 = make_classification(n_features=2, n_redundant=0, \
                                n_informative=1, random_state=1, n_clusters_per_class=1)
fig1, ax1 = plt.subplots();
ax1.scatter(X1[:,0],X1[:,1],c=y1, cmap='rainbow')
plt.show()
```



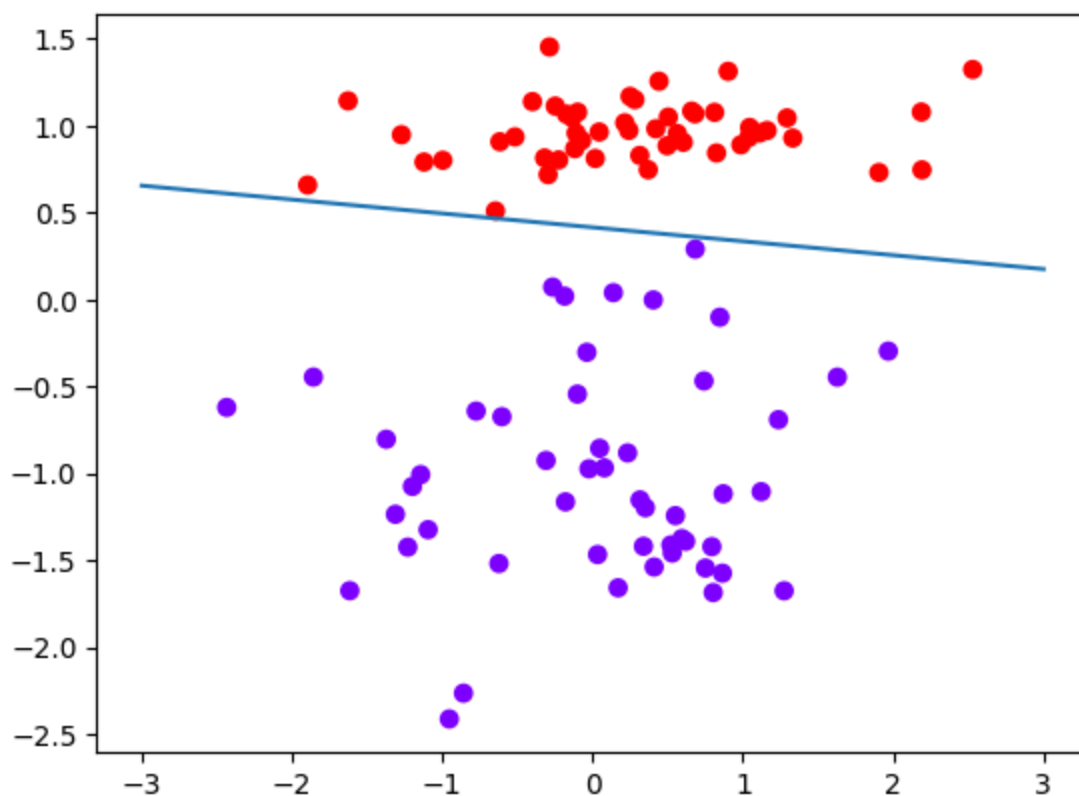
Notice that the two classes can clearly be separated by a single line. We can now fit a single layer neural network to the data.

```
In [ ]: nn1=Perceptron(alpha=1, max_iter=1000, fit_intercept=True)
        model=nn1.fit(X1,y1)
```

(Optional) Plot the line that the perceptron is modelling. Look at the documentation to find out how you can get the values of the weights and the bias.

```
In [ ]: xs= np.linspace(-3,3, 101)
        m = -nn1.coef_[0][0]/nn1.coef_[0][1]
        c = -nn1.intercept_/nn1.coef_[0][1]
        ys = m*xs+c
        ax1.plot(xs, ys)
        fig1
```

Out[]:

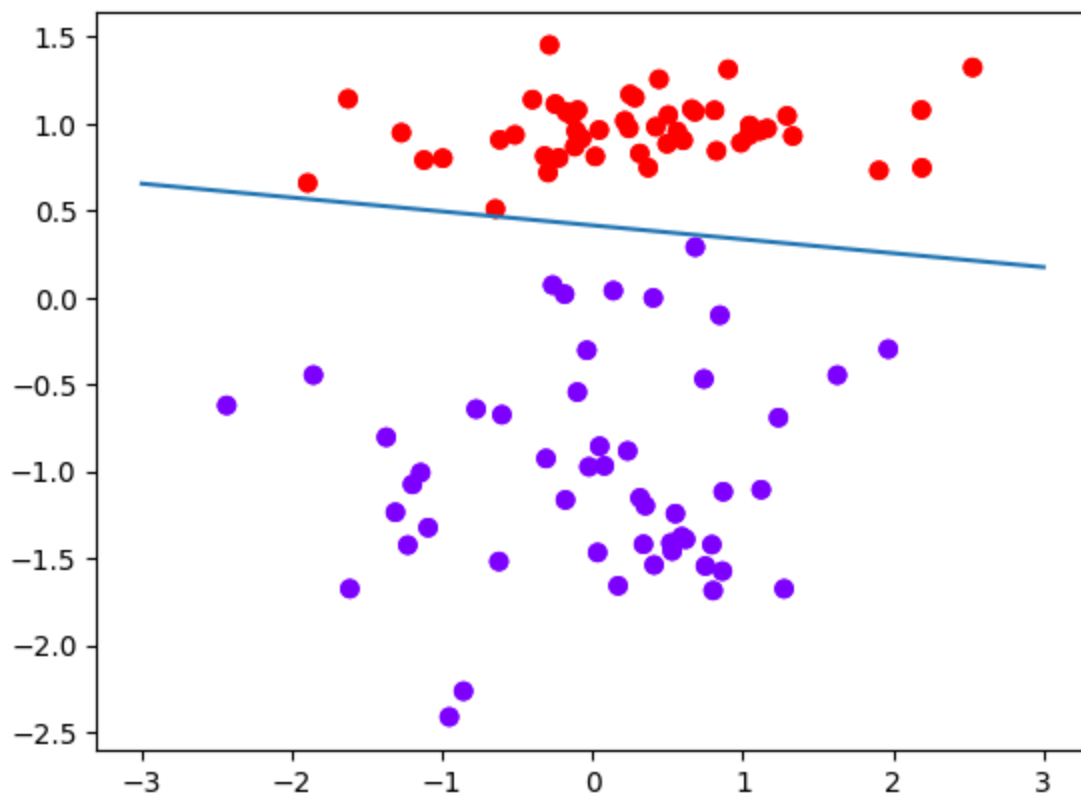


We can visualise the performance of this network again using a scatter plot, and colour the points using predicted class. Is the network able to give the right classification for each point?

```
In [ ]: ypred1=model.predict(X1)
# TODO: Make a scatter plot of the predictions.
ax1.scatter(X1[:,0],X1[:,1],c=ypred1, cmap='rainbow')
# How do the predictions compare with the ground truth?
from sklearn.metrics import accuracy_score
print (f'Accuracy is: {accuracy_score(y1, ypred1)}')
fig1
```

Accuracy is: 1.0

Out[]:

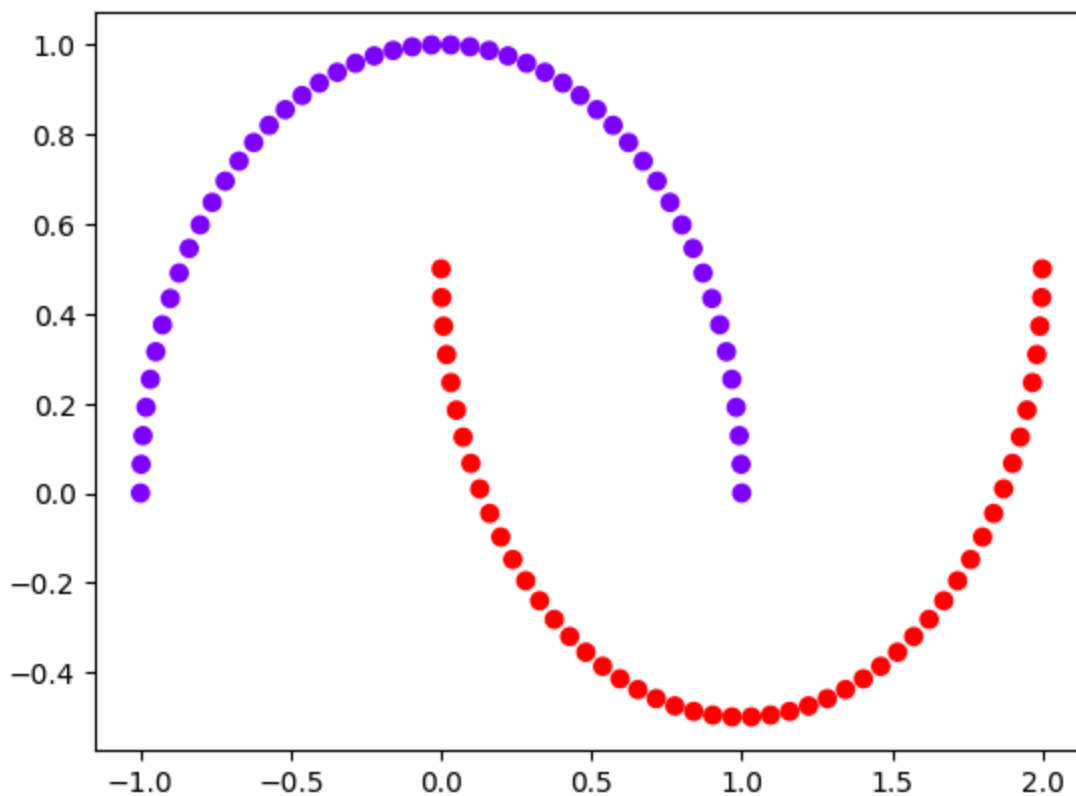


Multi-Layer NN on Toy Problems

In this question we consider two toy problems in which the classes are not linearly separable. In the first example the two classes form moon shapes.

```
In [ ]: X2, y2 = make_moons()  
fig2, ax2 = plt.subplots()  
ax2.scatter(X2[:,0],X2[:,1],c=y2, cmap='rainbow')
```

Out[]: <matplotlib.collections.PathCollection at 0x7e04200173e0>

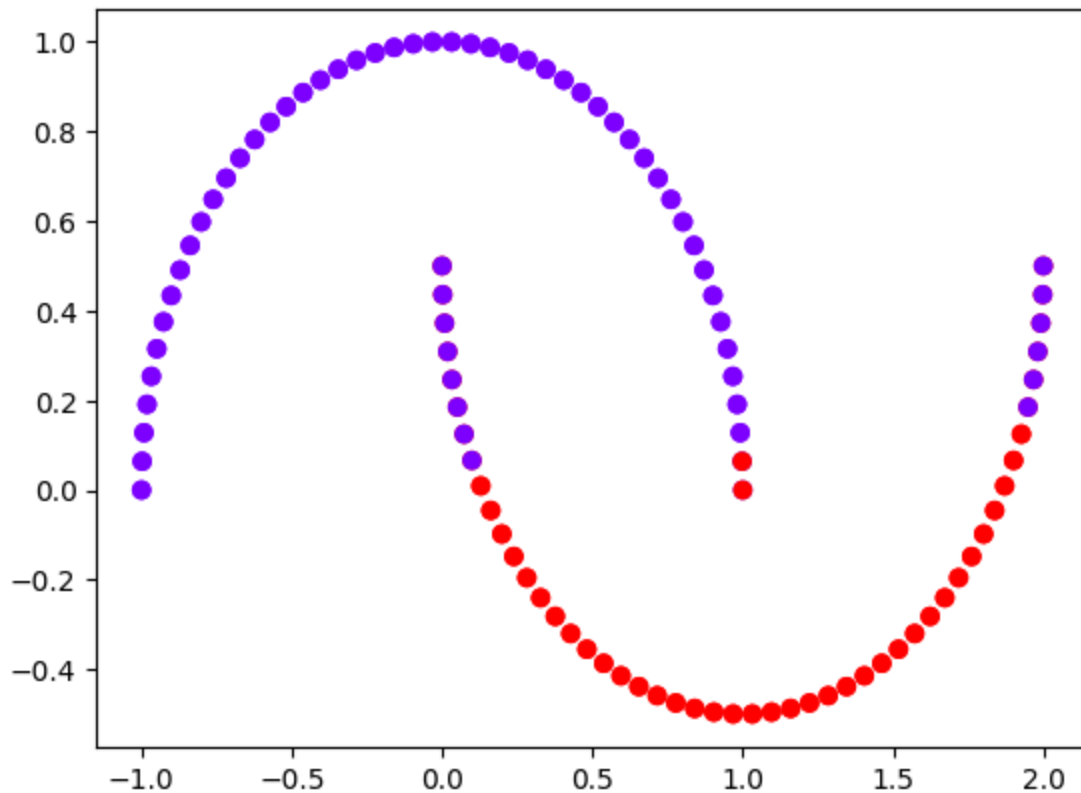


Try fitting a single-layer neural network to the data. Comment on the performance and hypothesise what might be the source of the errors.

```
In [ ]: nn2=Perceptron(alpha=1, max_iter=1000)
model2=nn2.fit(X2,y2)
ypred2=model2.predict(X2)
# TODO: Make a scatter plot of the predictions.
ax2.scatter(X2[:,0],X2[:,1],c=ypred2, cmap='rainbow')
# How do the predictions compare with the ground truth?
from sklearn.metrics import accuracy_score
print (f'Accuracy is: {accuracy_score(y2, ypred2)}')
fig2
```

Accuracy is: 0.84

Out[]:

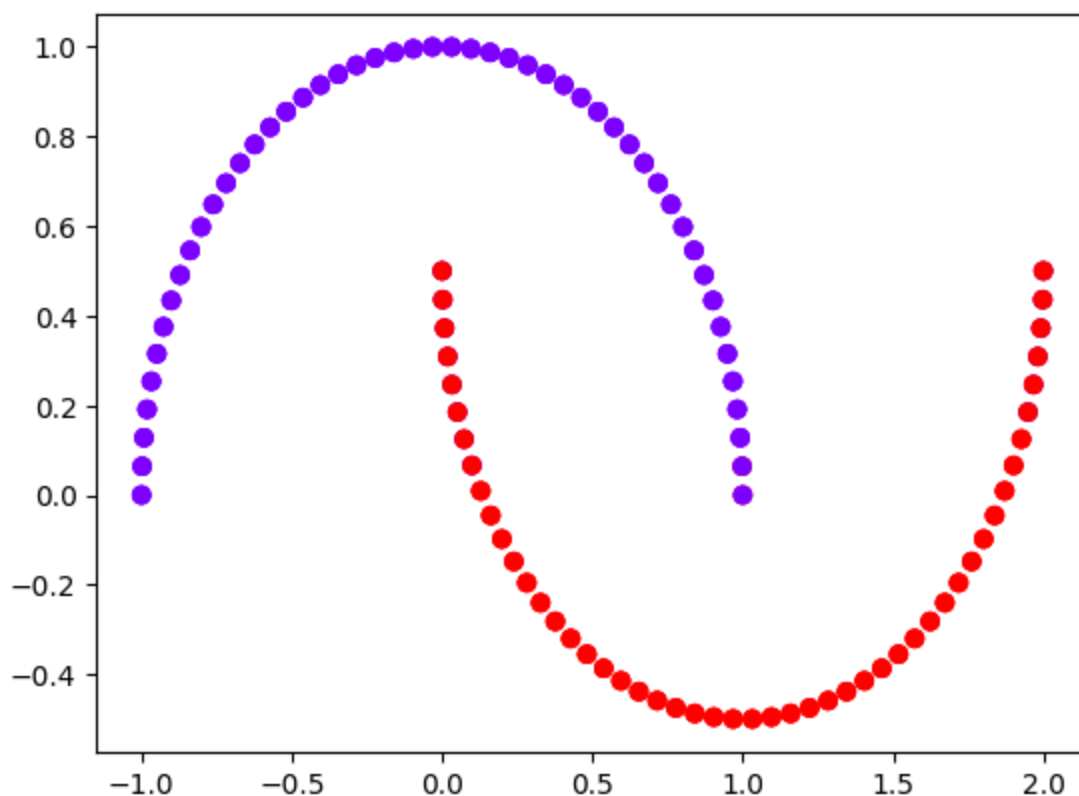


A single layer perceptron can only fit a single line to data, and this dataset can't be separated with a single line. Hence, we can't achieve perfect performance

We can now try to fit an multi-layer NN to the same data.

```
In [ ]: nn2=MLPClassifier(alpha=1,hidden_layer_sizes=(10,10,10,10), max_iter=1000)
model2=nn2.fit(X2,y2)
ypred2=model2.predict(X2)
ax2.scatter(X2[:,0],X2[:,1],c=ypred2, cmap='rainbow')
fig2
```

Out[]:



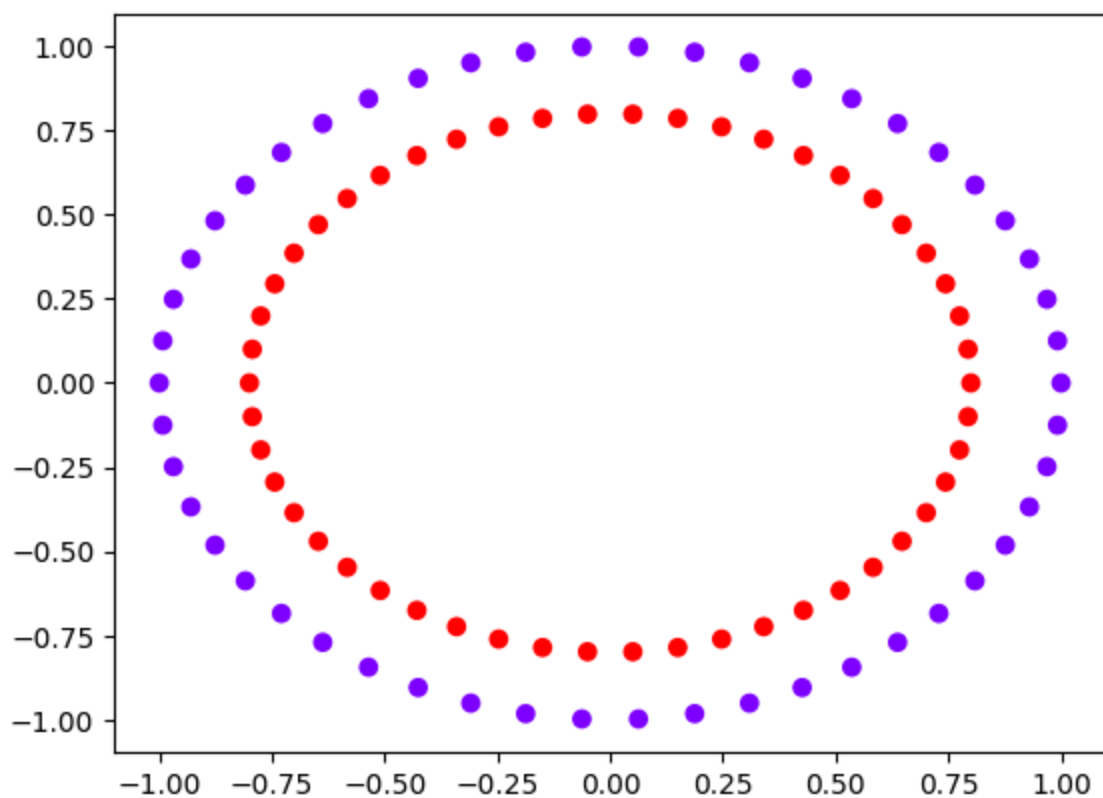
This NN has 4 hidden layers each with 10 neurons. The parameter `hidden_layer_sizes=(x,y,z, ...)` identifies the number of neurons in each layer. Try experimenting with different configurations of hidden layers to see the effect on performance.

Investigate the parameter `activation`. What are the different activation functions?

The activation functions are: 'identity', 'logistic', 'tanh', 'relu'

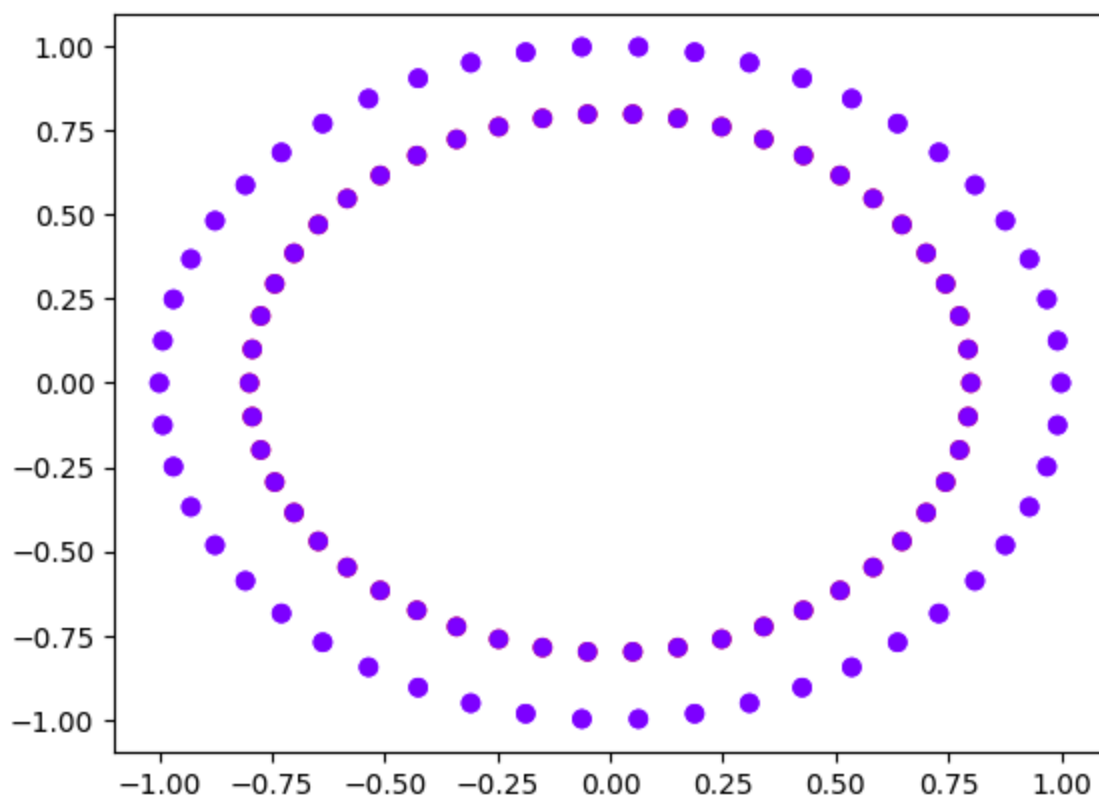
We now introduce another toy classification problem based on classes in the form of two circles. Experiment with using different NN architectures to fit this data and plot the results.

```
In [ ]: X3, y3 = make_circles()
fig3, ax3 = plt.subplots()
ax3.scatter(X3[:,0],X3[:,1],c=y3, cmap='rainbow')
plt.show()
```



```
In [ ]: nn3=MLPClassifier(alpha=1,hidden_layer_sizes=(10,100,100,10), max_iter=1000)
model3=nn3.fit(X3,y3)
ypred3=model3.predict(X3)
ax3.scatter(X3[:,0],X3[:,1],c=ypred3, cmap='rainbow')
fig3
```

Out[]:



Handwritten Digit Recognition

The data set contains images of hand-written digits. There are 10 classes where each class refers to a digit. Preprocessing programs were used to extract normalized bitmaps of handwritten digits from a preprinted form. 32×32 bitmaps are divided into non-overlapping blocks of 4×4 and the number of on pixels are counted in each block. This generates an input matrix of 8×8 where each element is an integer in the range 0...16.

```
In [ ]: from sklearn.model_selection import train_test_split
        from sklearn import metrics
        from sklearn.datasets import load_digits
        digits = load_digits()
```

Divide the data into training and test sets, fit a NN to the training data and evaluate its accuracy on both training and test data.

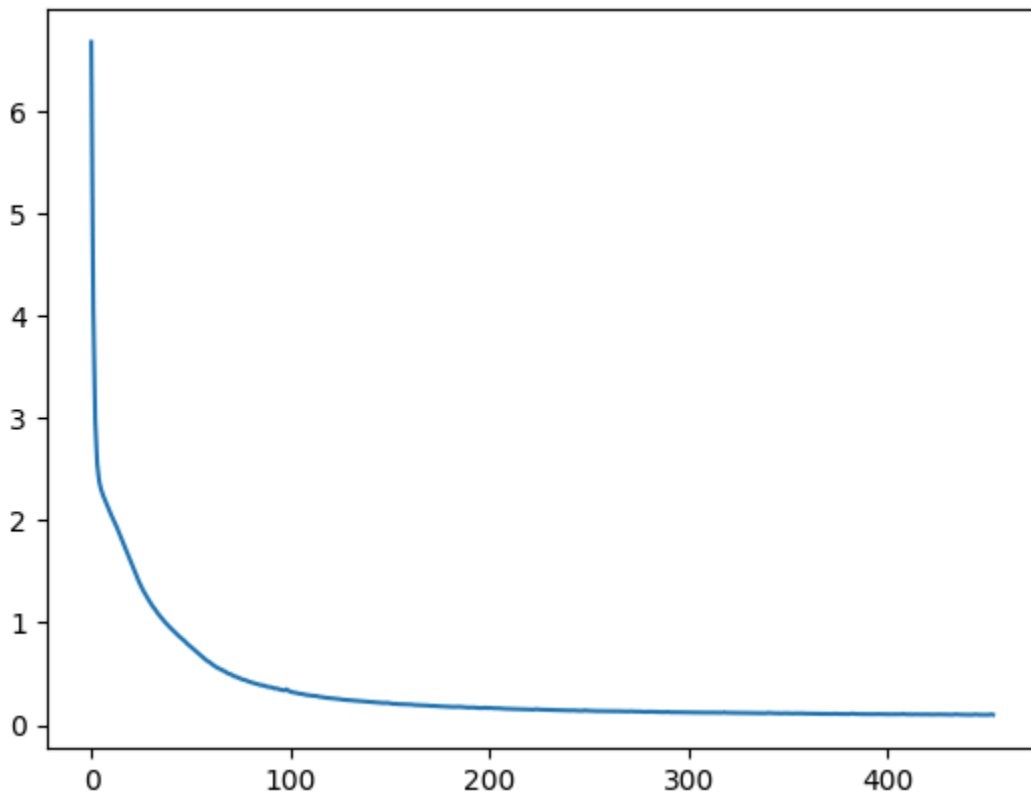
```
In [ ]: #TODO
        trainX, testX, trainy, testy = train_test_split(digits.data, digits.target, te
        nn4=MLPClassifier(alpha=1,hidden_layer_sizes=(10,10), max_iter=1000)
        model4=nn4.fit(trainX,trainy)
        train_pred=model4.predict(trainX)
        test_pred=model4.predict(testX)
```

```
print(f'Training Accuracy is: {accuracy_score(trainy, train_pred)}')  
print(f'Test Accuracy is: {accuracy_score(testy, test_pred)}')
```

Training Accuracy is: 0.9962894248608535
Test Accuracy is: 0.9666666666666667

It is also useful to plot a time series graph of the loss function (error) against iteration.

```
In [ ]: loss_values = model4.loss_curve_  
plt.figure()  
plt.plot(loss_values)  
plt.show()
```



Try experimenting with different architectures and activation functions.

Handwritten digit recognition in Pytorch

This is an excerpt from a tutorial in Pytorch, available at <https://docs.pytorch.org/tutorials/index.html>. Pytorch has lots of tutorials if you want to investigate.

In this tutorial, you are asked to:

1. Load a prebuilt dataset.

2. Build a neural network machine learning model that classifies images.
3. Train this neural network.
4. Evaluate the accuracy of the model.

We firstly import Pytorch

```
In [ ]: import torch
        print("PyTorch version:", torch.__version__)
```

PyTorch version: 2.9.0+cpu

Load the MNIST digits dataset

Load the dataset and convert data from integers into floating point numbers:

```
In [ ]: from torchvision.datasets import MNIST
        from torchvision.transforms import ToTensor
        import numpy as np

        # Load MNIST
        train_ds = MNIST(root="./data", train=True, download=True, transform=ToTensor())
        test_ds  = MNIST(root="./data", train=False, download=True, transform=ToTensor())

        # Convert to numpy (to match your existing code style)
        x_train = train_ds.data.numpy() / 255.0
        y_train = train_ds.targets.numpy()

        x_test  = test_ds.data.numpy() / 255.0
        y_test  = test_ds.targets.numpy()
```

```
100%|██████████| 9.91M/9.91M [00:01<00:00, 6.07MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 158kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.52MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.01MB/s]
```

Build a machine learning model

Build a Sequential model. Here, the model is a sequential model, meaning that it has a feed-forward architecture rather than a recurrent architecture or otherwise.

We set up a Sequential model and then add layers to it. Each layer can have different types: Flatten, Dense, Dropout. Look through the documentation for each layer to find out about them.

```
In [ ]: import torch.nn as nn

        model = nn.Sequential()
```

```

nn.Flatten(),          # (28, 28) -> 784
nn.Linear(28 * 28, 256),
nn.ReLU(),
nn.Dropout(0.2),
nn.Linear(256, 10)
)

```

For each datapoint, the model returns a vector of logits or log-odds scores, one for each class. Below, we make predictions for just one datapoint.

```

In [ ]: model.eval() # set eval mode (important for Dropout)

with torch.no_grad():
    x = torch.tensor(x_train[:1], dtype=torch.float32)
    predictions = model(x).numpy()

predictions

```

```

Out[ ]: array([[ -0.02873024,  0.07393599,  0.15608844, -0.07358637,  0.07487176,
                -0.05856224,  0.03719023, -0.05127727, -0.01412839,  0.05850118]],
              dtype=float32)

```

The `tf.nn.softmax` function converts these logits to probabilities for each class - now we have the probabilities that this datapoint belongs to each class.

```

In [ ]: torch.softmax(torch.tensor(predictions), dim=1).numpy()

Out[ ]: array([[0.09524962, 0.10554814, 0.11458532, 0.09107149, 0.10564697,
                0.0924501 , 0.10174009, 0.09312605, 0.09665064, 0.10393154]],
              dtype=float32)

```

For training the model, we need to define a loss function. We will use `losses.SparseCategoricalCrossentropy`, which takes a vector of logits and a True index and returns a scalar loss for each example.

```

In [ ]: loss_fn = nn.CrossEntropyLoss()

```

This loss is equal to the negative log probability of the true class: The loss is zero if the model is sure of the correct class.

This untrained model gives probabilities close to random (1/10 for each class), so the initial loss should be close to $-\text{tf.math.log}(1/10) \approx 2.3$.

```

In [ ]: y = torch.tensor(y_train[:1], dtype=torch.long)
        pred = torch.tensor(predictions, dtype=torch.float32)

        loss_fn(pred, y).item()

```

```

Out[ ]: 2.3810861110687256

```

Before you start training, we compile the model, setting the kind of optimizer we want to use (`adam`), the loss function, as set up above, and the metric we want to measure (`accuracy`).

```
In [ ]: optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def accuracy(logits, y):
    preds = torch.argmax(logits, dim=1)
    return (preds == y).float().mean().item()
```

Train and evaluate your model

```
In [ ]: model.train()

x = torch.tensor(x_train, dtype=torch.float32)
y = torch.tensor(y_train, dtype=torch.long)

epochs = 500
for epoch in range(epochs):
    logits = model(x)
    loss = loss_fn(logits, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    acc = (torch.argmax(logits, dim=1) == y).float().mean().item()
    print(f"Epoch {epoch+1}/{epochs} - loss: {loss.item():.4f} - acc: {acc:.4f}")
```

Epoch 1/500 - loss: 2.3031 - acc: 0.1077
Epoch 2/500 - loss: 2.2091 - acc: 0.3923
Epoch 3/500 - loss: 2.1194 - acc: 0.5808
Epoch 4/500 - loss: 2.0280 - acc: 0.6592
Epoch 5/500 - loss: 1.9333 - acc: 0.6944
Epoch 6/500 - loss: 1.8354 - acc: 0.7199
Epoch 7/500 - loss: 1.7359 - acc: 0.7406
Epoch 8/500 - loss: 1.6375 - acc: 0.7561
Epoch 9/500 - loss: 1.5400 - acc: 0.7671
Epoch 10/500 - loss: 1.4455 - acc: 0.7778
Epoch 11/500 - loss: 1.3561 - acc: 0.7842
Epoch 12/500 - loss: 1.2695 - acc: 0.7904
Epoch 13/500 - loss: 1.1883 - acc: 0.7965
Epoch 14/500 - loss: 1.1139 - acc: 0.8000
Epoch 15/500 - loss: 1.0443 - acc: 0.8037
Epoch 16/500 - loss: 0.9814 - acc: 0.8084
Epoch 17/500 - loss: 0.9218 - acc: 0.8142
Epoch 18/500 - loss: 0.8690 - acc: 0.8182
Epoch 19/500 - loss: 0.8223 - acc: 0.8213
Epoch 20/500 - loss: 0.7800 - acc: 0.8239
Epoch 21/500 - loss: 0.7386 - acc: 0.8285
Epoch 22/500 - loss: 0.7037 - acc: 0.8334
Epoch 23/500 - loss: 0.6720 - acc: 0.8370
Epoch 24/500 - loss: 0.6450 - acc: 0.8392
Epoch 25/500 - loss: 0.6194 - acc: 0.8429
Epoch 26/500 - loss: 0.5966 - acc: 0.8474
Epoch 27/500 - loss: 0.5760 - acc: 0.8499
Epoch 28/500 - loss: 0.5574 - acc: 0.8529
Epoch 29/500 - loss: 0.5395 - acc: 0.8562
Epoch 30/500 - loss: 0.5246 - acc: 0.8588
Epoch 31/500 - loss: 0.5101 - acc: 0.8622
Epoch 32/500 - loss: 0.4971 - acc: 0.8640
Epoch 33/500 - loss: 0.4859 - acc: 0.8669
Epoch 34/500 - loss: 0.4744 - acc: 0.8693
Epoch 35/500 - loss: 0.4637 - acc: 0.8702
Epoch 36/500 - loss: 0.4549 - acc: 0.8716
Epoch 37/500 - loss: 0.4451 - acc: 0.8744
Epoch 38/500 - loss: 0.4373 - acc: 0.8767
Epoch 39/500 - loss: 0.4309 - acc: 0.8786
Epoch 40/500 - loss: 0.4239 - acc: 0.8805
Epoch 41/500 - loss: 0.4160 - acc: 0.8836
Epoch 42/500 - loss: 0.4117 - acc: 0.8835
Epoch 43/500 - loss: 0.4042 - acc: 0.8853
Epoch 44/500 - loss: 0.3992 - acc: 0.8861
Epoch 45/500 - loss: 0.3946 - acc: 0.8875
Epoch 46/500 - loss: 0.3887 - acc: 0.8894
Epoch 47/500 - loss: 0.3830 - acc: 0.8909
Epoch 48/500 - loss: 0.3780 - acc: 0.8931
Epoch 49/500 - loss: 0.3750 - acc: 0.8930
Epoch 50/500 - loss: 0.3702 - acc: 0.8938
Epoch 51/500 - loss: 0.3668 - acc: 0.8954
Epoch 52/500 - loss: 0.3646 - acc: 0.8956
Epoch 53/500 - loss: 0.3587 - acc: 0.8974
Epoch 54/500 - loss: 0.3555 - acc: 0.8987

Epoch 55/500 - loss: 0.3519 - acc: 0.8994
Epoch 56/500 - loss: 0.3503 - acc: 0.8993
Epoch 57/500 - loss: 0.3468 - acc: 0.9003
Epoch 58/500 - loss: 0.3419 - acc: 0.9021
Epoch 59/500 - loss: 0.3402 - acc: 0.9023
Epoch 60/500 - loss: 0.3373 - acc: 0.9035
Epoch 61/500 - loss: 0.3332 - acc: 0.9038
Epoch 62/500 - loss: 0.3323 - acc: 0.9058
Epoch 63/500 - loss: 0.3291 - acc: 0.9064
Epoch 64/500 - loss: 0.3253 - acc: 0.9079
Epoch 65/500 - loss: 0.3251 - acc: 0.9074
Epoch 66/500 - loss: 0.3219 - acc: 0.9085
Epoch 67/500 - loss: 0.3188 - acc: 0.9092
Epoch 68/500 - loss: 0.3160 - acc: 0.9100
Epoch 69/500 - loss: 0.3145 - acc: 0.9110
Epoch 70/500 - loss: 0.3117 - acc: 0.9116
Epoch 71/500 - loss: 0.3092 - acc: 0.9116
Epoch 72/500 - loss: 0.3081 - acc: 0.9123
Epoch 73/500 - loss: 0.3048 - acc: 0.9144
Epoch 74/500 - loss: 0.3052 - acc: 0.9132
Epoch 75/500 - loss: 0.3008 - acc: 0.9149
Epoch 76/500 - loss: 0.2988 - acc: 0.9157
Epoch 77/500 - loss: 0.2967 - acc: 0.9163
Epoch 78/500 - loss: 0.2951 - acc: 0.9156
Epoch 79/500 - loss: 0.2944 - acc: 0.9163
Epoch 80/500 - loss: 0.2911 - acc: 0.9176
Epoch 81/500 - loss: 0.2905 - acc: 0.9182
Epoch 82/500 - loss: 0.2874 - acc: 0.9191
Epoch 83/500 - loss: 0.2865 - acc: 0.9192
Epoch 84/500 - loss: 0.2846 - acc: 0.9196
Epoch 85/500 - loss: 0.2821 - acc: 0.9202
Epoch 86/500 - loss: 0.2813 - acc: 0.9215
Epoch 87/500 - loss: 0.2794 - acc: 0.9208
Epoch 88/500 - loss: 0.2771 - acc: 0.9213
Epoch 89/500 - loss: 0.2749 - acc: 0.9226
Epoch 90/500 - loss: 0.2727 - acc: 0.9236
Epoch 91/500 - loss: 0.2719 - acc: 0.9227
Epoch 92/500 - loss: 0.2697 - acc: 0.9248
Epoch 93/500 - loss: 0.2695 - acc: 0.9244
Epoch 94/500 - loss: 0.2663 - acc: 0.9258
Epoch 95/500 - loss: 0.2652 - acc: 0.9261
Epoch 96/500 - loss: 0.2632 - acc: 0.9268
Epoch 97/500 - loss: 0.2628 - acc: 0.9264
Epoch 98/500 - loss: 0.2600 - acc: 0.9269
Epoch 99/500 - loss: 0.2595 - acc: 0.9273
Epoch 100/500 - loss: 0.2579 - acc: 0.9265
Epoch 101/500 - loss: 0.2564 - acc: 0.9277
Epoch 102/500 - loss: 0.2554 - acc: 0.9275
Epoch 103/500 - loss: 0.2526 - acc: 0.9288
Epoch 104/500 - loss: 0.2516 - acc: 0.9295
Epoch 105/500 - loss: 0.2494 - acc: 0.9303
Epoch 106/500 - loss: 0.2487 - acc: 0.9301
Epoch 107/500 - loss: 0.2475 - acc: 0.9303
Epoch 108/500 - loss: 0.2458 - acc: 0.9301

Epoch 109/500 - loss: 0.2439 - acc: 0.9316
Epoch 110/500 - loss: 0.2426 - acc: 0.9312
Epoch 111/500 - loss: 0.2413 - acc: 0.9325
Epoch 112/500 - loss: 0.2390 - acc: 0.9335
Epoch 113/500 - loss: 0.2374 - acc: 0.9344
Epoch 114/500 - loss: 0.2371 - acc: 0.9337
Epoch 115/500 - loss: 0.2351 - acc: 0.9340
Epoch 116/500 - loss: 0.2341 - acc: 0.9353
Epoch 117/500 - loss: 0.2323 - acc: 0.9351
Epoch 118/500 - loss: 0.2306 - acc: 0.9349
Epoch 119/500 - loss: 0.2302 - acc: 0.9352
Epoch 120/500 - loss: 0.2290 - acc: 0.9359
Epoch 121/500 - loss: 0.2274 - acc: 0.9365
Epoch 122/500 - loss: 0.2256 - acc: 0.9367
Epoch 123/500 - loss: 0.2243 - acc: 0.9370
Epoch 124/500 - loss: 0.2220 - acc: 0.9377
Epoch 125/500 - loss: 0.2215 - acc: 0.9371
Epoch 126/500 - loss: 0.2206 - acc: 0.9377
Epoch 127/500 - loss: 0.2193 - acc: 0.9392
Epoch 128/500 - loss: 0.2173 - acc: 0.9392
Epoch 129/500 - loss: 0.2157 - acc: 0.9395
Epoch 130/500 - loss: 0.2159 - acc: 0.9404
Epoch 131/500 - loss: 0.2144 - acc: 0.9398
Epoch 132/500 - loss: 0.2144 - acc: 0.9406
Epoch 133/500 - loss: 0.2107 - acc: 0.9420
Epoch 134/500 - loss: 0.2107 - acc: 0.9407
Epoch 135/500 - loss: 0.2088 - acc: 0.9413
Epoch 136/500 - loss: 0.2082 - acc: 0.9414
Epoch 137/500 - loss: 0.2080 - acc: 0.9416
Epoch 138/500 - loss: 0.2056 - acc: 0.9420
Epoch 139/500 - loss: 0.2037 - acc: 0.9432
Epoch 140/500 - loss: 0.2041 - acc: 0.9430
Epoch 141/500 - loss: 0.2025 - acc: 0.9428
Epoch 142/500 - loss: 0.2011 - acc: 0.9439
Epoch 143/500 - loss: 0.1998 - acc: 0.9448
Epoch 144/500 - loss: 0.1987 - acc: 0.9446
Epoch 145/500 - loss: 0.1974 - acc: 0.9452
Epoch 146/500 - loss: 0.1965 - acc: 0.9452
Epoch 147/500 - loss: 0.1957 - acc: 0.9455
Epoch 148/500 - loss: 0.1946 - acc: 0.9449
Epoch 149/500 - loss: 0.1939 - acc: 0.9458
Epoch 150/500 - loss: 0.1934 - acc: 0.9461
Epoch 151/500 - loss: 0.1915 - acc: 0.9466
Epoch 152/500 - loss: 0.1904 - acc: 0.9464
Epoch 153/500 - loss: 0.1903 - acc: 0.9462
Epoch 154/500 - loss: 0.1884 - acc: 0.9476
Epoch 155/500 - loss: 0.1874 - acc: 0.9474
Epoch 156/500 - loss: 0.1866 - acc: 0.9470
Epoch 157/500 - loss: 0.1848 - acc: 0.9482
Epoch 158/500 - loss: 0.1849 - acc: 0.9478
Epoch 159/500 - loss: 0.1836 - acc: 0.9485
Epoch 160/500 - loss: 0.1828 - acc: 0.9488
Epoch 161/500 - loss: 0.1820 - acc: 0.9487
Epoch 162/500 - loss: 0.1802 - acc: 0.9490

Epoch	163/500	-	loss:	0.1807	-	acc:	0.9489
Epoch	164/500	-	loss:	0.1796	-	acc:	0.9498
Epoch	165/500	-	loss:	0.1776	-	acc:	0.9498
Epoch	166/500	-	loss:	0.1781	-	acc:	0.9504
Epoch	167/500	-	loss:	0.1758	-	acc:	0.9506
Epoch	168/500	-	loss:	0.1757	-	acc:	0.9506
Epoch	169/500	-	loss:	0.1750	-	acc:	0.9509
Epoch	170/500	-	loss:	0.1732	-	acc:	0.9513
Epoch	171/500	-	loss:	0.1734	-	acc:	0.9514
Epoch	172/500	-	loss:	0.1718	-	acc:	0.9525
Epoch	173/500	-	loss:	0.1718	-	acc:	0.9516
Epoch	174/500	-	loss:	0.1706	-	acc:	0.9521
Epoch	175/500	-	loss:	0.1692	-	acc:	0.9527
Epoch	176/500	-	loss:	0.1683	-	acc:	0.9528
Epoch	177/500	-	loss:	0.1675	-	acc:	0.9524
Epoch	178/500	-	loss:	0.1677	-	acc:	0.9530
Epoch	179/500	-	loss:	0.1659	-	acc:	0.9526
Epoch	180/500	-	loss:	0.1658	-	acc:	0.9531
Epoch	181/500	-	loss:	0.1647	-	acc:	0.9535
Epoch	182/500	-	loss:	0.1634	-	acc:	0.9547
Epoch	183/500	-	loss:	0.1641	-	acc:	0.9539
Epoch	184/500	-	loss:	0.1629	-	acc:	0.9542
Epoch	185/500	-	loss:	0.1620	-	acc:	0.9544
Epoch	186/500	-	loss:	0.1617	-	acc:	0.9538
Epoch	187/500	-	loss:	0.1598	-	acc:	0.9546
Epoch	188/500	-	loss:	0.1603	-	acc:	0.9548
Epoch	189/500	-	loss:	0.1583	-	acc:	0.9554
Epoch	190/500	-	loss:	0.1582	-	acc:	0.9552
Epoch	191/500	-	loss:	0.1568	-	acc:	0.9554
Epoch	192/500	-	loss:	0.1564	-	acc:	0.9558
Epoch	193/500	-	loss:	0.1556	-	acc:	0.9565
Epoch	194/500	-	loss:	0.1555	-	acc:	0.9562
Epoch	195/500	-	loss:	0.1539	-	acc:	0.9571
Epoch	196/500	-	loss:	0.1533	-	acc:	0.9564
Epoch	197/500	-	loss:	0.1538	-	acc:	0.9564
Epoch	198/500	-	loss:	0.1522	-	acc:	0.9568
Epoch	199/500	-	loss:	0.1516	-	acc:	0.9576
Epoch	200/500	-	loss:	0.1510	-	acc:	0.9581
Epoch	201/500	-	loss:	0.1498	-	acc:	0.9584
Epoch	202/500	-	loss:	0.1505	-	acc:	0.9575
Epoch	203/500	-	loss:	0.1484	-	acc:	0.9588
Epoch	204/500	-	loss:	0.1478	-	acc:	0.9577
Epoch	205/500	-	loss:	0.1478	-	acc:	0.9587
Epoch	206/500	-	loss:	0.1472	-	acc:	0.9588
Epoch	207/500	-	loss:	0.1474	-	acc:	0.9587
Epoch	208/500	-	loss:	0.1455	-	acc:	0.9594
Epoch	209/500	-	loss:	0.1447	-	acc:	0.9596
Epoch	210/500	-	loss:	0.1443	-	acc:	0.9587
Epoch	211/500	-	loss:	0.1446	-	acc:	0.9592
Epoch	212/500	-	loss:	0.1444	-	acc:	0.9596
Epoch	213/500	-	loss:	0.1424	-	acc:	0.9600
Epoch	214/500	-	loss:	0.1418	-	acc:	0.9600
Epoch	215/500	-	loss:	0.1409	-	acc:	0.9600
Epoch	216/500	-	loss:	0.1411	-	acc:	0.9607

Epoch 217/500 - loss: 0.1401 - acc: 0.9600
Epoch 218/500 - loss: 0.1405 - acc: 0.9611
Epoch 219/500 - loss: 0.1392 - acc: 0.9613
Epoch 220/500 - loss: 0.1383 - acc: 0.9610
Epoch 221/500 - loss: 0.1381 - acc: 0.9610
Epoch 222/500 - loss: 0.1367 - acc: 0.9614
Epoch 223/500 - loss: 0.1366 - acc: 0.9617
Epoch 224/500 - loss: 0.1346 - acc: 0.9623
Epoch 225/500 - loss: 0.1357 - acc: 0.9618
Epoch 226/500 - loss: 0.1342 - acc: 0.9627
Epoch 227/500 - loss: 0.1346 - acc: 0.9625
Epoch 228/500 - loss: 0.1347 - acc: 0.9618
Epoch 229/500 - loss: 0.1335 - acc: 0.9631
Epoch 230/500 - loss: 0.1327 - acc: 0.9630
Epoch 231/500 - loss: 0.1328 - acc: 0.9620
Epoch 232/500 - loss: 0.1312 - acc: 0.9626
Epoch 233/500 - loss: 0.1320 - acc: 0.9630
Epoch 234/500 - loss: 0.1307 - acc: 0.9635
Epoch 235/500 - loss: 0.1302 - acc: 0.9637
Epoch 236/500 - loss: 0.1303 - acc: 0.9635
Epoch 237/500 - loss: 0.1290 - acc: 0.9643
Epoch 238/500 - loss: 0.1294 - acc: 0.9639
Epoch 239/500 - loss: 0.1272 - acc: 0.9641
Epoch 240/500 - loss: 0.1283 - acc: 0.9642
Epoch 241/500 - loss: 0.1284 - acc: 0.9635
Epoch 242/500 - loss: 0.1280 - acc: 0.9634
Epoch 243/500 - loss: 0.1262 - acc: 0.9639
Epoch 244/500 - loss: 0.1258 - acc: 0.9649
Epoch 245/500 - loss: 0.1240 - acc: 0.9653
Epoch 246/500 - loss: 0.1248 - acc: 0.9645
Epoch 247/500 - loss: 0.1252 - acc: 0.9643
Epoch 248/500 - loss: 0.1245 - acc: 0.9650
Epoch 249/500 - loss: 0.1236 - acc: 0.9650
Epoch 250/500 - loss: 0.1236 - acc: 0.9651
Epoch 251/500 - loss: 0.1233 - acc: 0.9651
Epoch 252/500 - loss: 0.1220 - acc: 0.9658
Epoch 253/500 - loss: 0.1219 - acc: 0.9657
Epoch 254/500 - loss: 0.1211 - acc: 0.9661
Epoch 255/500 - loss: 0.1219 - acc: 0.9655
Epoch 256/500 - loss: 0.1195 - acc: 0.9663
Epoch 257/500 - loss: 0.1192 - acc: 0.9663
Epoch 258/500 - loss: 0.1200 - acc: 0.9656
Epoch 259/500 - loss: 0.1197 - acc: 0.9659
Epoch 260/500 - loss: 0.1184 - acc: 0.9670
Epoch 261/500 - loss: 0.1187 - acc: 0.9667
Epoch 262/500 - loss: 0.1175 - acc: 0.9668
Epoch 263/500 - loss: 0.1176 - acc: 0.9668
Epoch 264/500 - loss: 0.1163 - acc: 0.9673
Epoch 265/500 - loss: 0.1165 - acc: 0.9671
Epoch 266/500 - loss: 0.1161 - acc: 0.9672
Epoch 267/500 - loss: 0.1151 - acc: 0.9685
Epoch 268/500 - loss: 0.1154 - acc: 0.9680
Epoch 269/500 - loss: 0.1148 - acc: 0.9675
Epoch 270/500 - loss: 0.1156 - acc: 0.9671

Epoch 271/500 - loss: 0.1134 - acc: 0.9679
Epoch 272/500 - loss: 0.1135 - acc: 0.9683
Epoch 273/500 - loss: 0.1129 - acc: 0.9683
Epoch 274/500 - loss: 0.1130 - acc: 0.9681
Epoch 275/500 - loss: 0.1110 - acc: 0.9691
Epoch 276/500 - loss: 0.1114 - acc: 0.9689
Epoch 277/500 - loss: 0.1125 - acc: 0.9682
Epoch 278/500 - loss: 0.1103 - acc: 0.9693
Epoch 279/500 - loss: 0.1100 - acc: 0.9693
Epoch 280/500 - loss: 0.1108 - acc: 0.9688
Epoch 281/500 - loss: 0.1094 - acc: 0.9691
Epoch 282/500 - loss: 0.1087 - acc: 0.9693
Epoch 283/500 - loss: 0.1094 - acc: 0.9693
Epoch 284/500 - loss: 0.1080 - acc: 0.9699
Epoch 285/500 - loss: 0.1074 - acc: 0.9695
Epoch 286/500 - loss: 0.1089 - acc: 0.9694
Epoch 287/500 - loss: 0.1076 - acc: 0.9692
Epoch 288/500 - loss: 0.1068 - acc: 0.9702
Epoch 289/500 - loss: 0.1076 - acc: 0.9696
Epoch 290/500 - loss: 0.1066 - acc: 0.9696
Epoch 291/500 - loss: 0.1051 - acc: 0.9708
Epoch 292/500 - loss: 0.1062 - acc: 0.9704
Epoch 293/500 - loss: 0.1050 - acc: 0.9703
Epoch 294/500 - loss: 0.1042 - acc: 0.9703
Epoch 295/500 - loss: 0.1046 - acc: 0.9709
Epoch 296/500 - loss: 0.1052 - acc: 0.9706
Epoch 297/500 - loss: 0.1036 - acc: 0.9711
Epoch 298/500 - loss: 0.1026 - acc: 0.9711
Epoch 299/500 - loss: 0.1034 - acc: 0.9710
Epoch 300/500 - loss: 0.1033 - acc: 0.9711
Epoch 301/500 - loss: 0.1024 - acc: 0.9716
Epoch 302/500 - loss: 0.1030 - acc: 0.9713
Epoch 303/500 - loss: 0.1023 - acc: 0.9715
Epoch 304/500 - loss: 0.1014 - acc: 0.9713
Epoch 305/500 - loss: 0.1008 - acc: 0.9718
Epoch 306/500 - loss: 0.1013 - acc: 0.9718
Epoch 307/500 - loss: 0.1009 - acc: 0.9714
Epoch 308/500 - loss: 0.1001 - acc: 0.9716
Epoch 309/500 - loss: 0.0991 - acc: 0.9723
Epoch 310/500 - loss: 0.0987 - acc: 0.9720
Epoch 311/500 - loss: 0.0988 - acc: 0.9723
Epoch 312/500 - loss: 0.0986 - acc: 0.9720
Epoch 313/500 - loss: 0.0983 - acc: 0.9723
Epoch 314/500 - loss: 0.0989 - acc: 0.9723
Epoch 315/500 - loss: 0.0981 - acc: 0.9724
Epoch 316/500 - loss: 0.0985 - acc: 0.9727
Epoch 317/500 - loss: 0.0969 - acc: 0.9728
Epoch 318/500 - loss: 0.0976 - acc: 0.9724
Epoch 319/500 - loss: 0.0969 - acc: 0.9729
Epoch 320/500 - loss: 0.0957 - acc: 0.9732
Epoch 321/500 - loss: 0.0957 - acc: 0.9729
Epoch 322/500 - loss: 0.0965 - acc: 0.9728
Epoch 323/500 - loss: 0.0947 - acc: 0.9736
Epoch 324/500 - loss: 0.0962 - acc: 0.9728

Epoch 325/500 - loss: 0.0943 - acc: 0.9738
Epoch 326/500 - loss: 0.0952 - acc: 0.9740
Epoch 327/500 - loss: 0.0942 - acc: 0.9737
Epoch 328/500 - loss: 0.0938 - acc: 0.9738
Epoch 329/500 - loss: 0.0942 - acc: 0.9741
Epoch 330/500 - loss: 0.0929 - acc: 0.9736
Epoch 331/500 - loss: 0.0921 - acc: 0.9743
Epoch 332/500 - loss: 0.0922 - acc: 0.9740
Epoch 333/500 - loss: 0.0927 - acc: 0.9741
Epoch 334/500 - loss: 0.0911 - acc: 0.9747
Epoch 335/500 - loss: 0.0926 - acc: 0.9735
Epoch 336/500 - loss: 0.0918 - acc: 0.9743
Epoch 337/500 - loss: 0.0911 - acc: 0.9743
Epoch 338/500 - loss: 0.0910 - acc: 0.9747
Epoch 339/500 - loss: 0.0912 - acc: 0.9740
Epoch 340/500 - loss: 0.0910 - acc: 0.9741
Epoch 341/500 - loss: 0.0897 - acc: 0.9750
Epoch 342/500 - loss: 0.0899 - acc: 0.9750
Epoch 343/500 - loss: 0.0894 - acc: 0.9744
Epoch 344/500 - loss: 0.0894 - acc: 0.9746
Epoch 345/500 - loss: 0.0883 - acc: 0.9750
Epoch 346/500 - loss: 0.0890 - acc: 0.9750
Epoch 347/500 - loss: 0.0880 - acc: 0.9743
Epoch 348/500 - loss: 0.0882 - acc: 0.9752
Epoch 349/500 - loss: 0.0874 - acc: 0.9755
Epoch 350/500 - loss: 0.0871 - acc: 0.9753
Epoch 351/500 - loss: 0.0877 - acc: 0.9750
Epoch 352/500 - loss: 0.0880 - acc: 0.9753
Epoch 353/500 - loss: 0.0869 - acc: 0.9759
Epoch 354/500 - loss: 0.0865 - acc: 0.9758
Epoch 355/500 - loss: 0.0850 - acc: 0.9763
Epoch 356/500 - loss: 0.0858 - acc: 0.9758
Epoch 357/500 - loss: 0.0860 - acc: 0.9757
Epoch 358/500 - loss: 0.0853 - acc: 0.9755
Epoch 359/500 - loss: 0.0852 - acc: 0.9760
Epoch 360/500 - loss: 0.0847 - acc: 0.9757
Epoch 361/500 - loss: 0.0844 - acc: 0.9764
Epoch 362/500 - loss: 0.0844 - acc: 0.9763
Epoch 363/500 - loss: 0.0843 - acc: 0.9760
Epoch 364/500 - loss: 0.0843 - acc: 0.9760
Epoch 365/500 - loss: 0.0845 - acc: 0.9761
Epoch 366/500 - loss: 0.0839 - acc: 0.9765
Epoch 367/500 - loss: 0.0836 - acc: 0.9761
Epoch 368/500 - loss: 0.0837 - acc: 0.9762
Epoch 369/500 - loss: 0.0818 - acc: 0.9776
Epoch 370/500 - loss: 0.0827 - acc: 0.9765
Epoch 371/500 - loss: 0.0820 - acc: 0.9769
Epoch 372/500 - loss: 0.0826 - acc: 0.9759
Epoch 373/500 - loss: 0.0809 - acc: 0.9774
Epoch 374/500 - loss: 0.0825 - acc: 0.9764
Epoch 375/500 - loss: 0.0827 - acc: 0.9764
Epoch 376/500 - loss: 0.0811 - acc: 0.9769
Epoch 377/500 - loss: 0.0811 - acc: 0.9775
Epoch 378/500 - loss: 0.0805 - acc: 0.9771

Epoch 379/500 - loss: 0.0804 - acc: 0.9778
Epoch 380/500 - loss: 0.0795 - acc: 0.9780
Epoch 381/500 - loss: 0.0799 - acc: 0.9775
Epoch 382/500 - loss: 0.0799 - acc: 0.9777
Epoch 383/500 - loss: 0.0800 - acc: 0.9773
Epoch 384/500 - loss: 0.0789 - acc: 0.9782
Epoch 385/500 - loss: 0.0784 - acc: 0.9777
Epoch 386/500 - loss: 0.0793 - acc: 0.9772
Epoch 387/500 - loss: 0.0789 - acc: 0.9772
Epoch 388/500 - loss: 0.0790 - acc: 0.9775
Epoch 389/500 - loss: 0.0782 - acc: 0.9779
Epoch 390/500 - loss: 0.0775 - acc: 0.9776
Epoch 391/500 - loss: 0.0773 - acc: 0.9780
Epoch 392/500 - loss: 0.0774 - acc: 0.9780
Epoch 393/500 - loss: 0.0778 - acc: 0.9783
Epoch 394/500 - loss: 0.0773 - acc: 0.9776
Epoch 395/500 - loss: 0.0773 - acc: 0.9783
Epoch 396/500 - loss: 0.0766 - acc: 0.9786
Epoch 397/500 - loss: 0.0766 - acc: 0.9779
Epoch 398/500 - loss: 0.0767 - acc: 0.9783
Epoch 399/500 - loss: 0.0751 - acc: 0.9792
Epoch 400/500 - loss: 0.0751 - acc: 0.9791
Epoch 401/500 - loss: 0.0757 - acc: 0.9788
Epoch 402/500 - loss: 0.0765 - acc: 0.9779
Epoch 403/500 - loss: 0.0762 - acc: 0.9783
Epoch 404/500 - loss: 0.0743 - acc: 0.9791
Epoch 405/500 - loss: 0.0747 - acc: 0.9786
Epoch 406/500 - loss: 0.0746 - acc: 0.9791
Epoch 407/500 - loss: 0.0731 - acc: 0.9794
Epoch 408/500 - loss: 0.0739 - acc: 0.9786
Epoch 409/500 - loss: 0.0748 - acc: 0.9789
Epoch 410/500 - loss: 0.0738 - acc: 0.9791
Epoch 411/500 - loss: 0.0729 - acc: 0.9788
Epoch 412/500 - loss: 0.0722 - acc: 0.9798
Epoch 413/500 - loss: 0.0731 - acc: 0.9794
Epoch 414/500 - loss: 0.0734 - acc: 0.9788
Epoch 415/500 - loss: 0.0732 - acc: 0.9790
Epoch 416/500 - loss: 0.0723 - acc: 0.9793
Epoch 417/500 - loss: 0.0717 - acc: 0.9800
Epoch 418/500 - loss: 0.0722 - acc: 0.9793
Epoch 419/500 - loss: 0.0713 - acc: 0.9798
Epoch 420/500 - loss: 0.0697 - acc: 0.9804
Epoch 421/500 - loss: 0.0708 - acc: 0.9804
Epoch 422/500 - loss: 0.0705 - acc: 0.9805
Epoch 423/500 - loss: 0.0702 - acc: 0.9805
Epoch 424/500 - loss: 0.0712 - acc: 0.9794
Epoch 425/500 - loss: 0.0707 - acc: 0.9797
Epoch 426/500 - loss: 0.0704 - acc: 0.9802
Epoch 427/500 - loss: 0.0702 - acc: 0.9802
Epoch 428/500 - loss: 0.0698 - acc: 0.9803
Epoch 429/500 - loss: 0.0698 - acc: 0.9804
Epoch 430/500 - loss: 0.0697 - acc: 0.9802
Epoch 431/500 - loss: 0.0686 - acc: 0.9804
Epoch 432/500 - loss: 0.0702 - acc: 0.9797

Epoch 433/500 - loss: 0.0690 - acc: 0.9807
Epoch 434/500 - loss: 0.0696 - acc: 0.9808
Epoch 435/500 - loss: 0.0694 - acc: 0.9800
Epoch 436/500 - loss: 0.0678 - acc: 0.9813
Epoch 437/500 - loss: 0.0697 - acc: 0.9798
Epoch 438/500 - loss: 0.0686 - acc: 0.9802
Epoch 439/500 - loss: 0.0677 - acc: 0.9807
Epoch 440/500 - loss: 0.0677 - acc: 0.9810
Epoch 441/500 - loss: 0.0678 - acc: 0.9808
Epoch 442/500 - loss: 0.0677 - acc: 0.9810
Epoch 443/500 - loss: 0.0673 - acc: 0.9811
Epoch 444/500 - loss: 0.0676 - acc: 0.9813
Epoch 445/500 - loss: 0.0674 - acc: 0.9810
Epoch 446/500 - loss: 0.0660 - acc: 0.9816
Epoch 447/500 - loss: 0.0673 - acc: 0.9809
Epoch 448/500 - loss: 0.0658 - acc: 0.9813
Epoch 449/500 - loss: 0.0669 - acc: 0.9813
Epoch 450/500 - loss: 0.0654 - acc: 0.9816
Epoch 451/500 - loss: 0.0664 - acc: 0.9808
Epoch 452/500 - loss: 0.0655 - acc: 0.9817
Epoch 453/500 - loss: 0.0651 - acc: 0.9812
Epoch 454/500 - loss: 0.0651 - acc: 0.9816
Epoch 455/500 - loss: 0.0654 - acc: 0.9815
Epoch 456/500 - loss: 0.0657 - acc: 0.9812
Epoch 457/500 - loss: 0.0645 - acc: 0.9815
Epoch 458/500 - loss: 0.0646 - acc: 0.9816
Epoch 459/500 - loss: 0.0649 - acc: 0.9812
Epoch 460/500 - loss: 0.0641 - acc: 0.9817
Epoch 461/500 - loss: 0.0645 - acc: 0.9818
Epoch 462/500 - loss: 0.0636 - acc: 0.9823
Epoch 463/500 - loss: 0.0636 - acc: 0.9822
Epoch 464/500 - loss: 0.0639 - acc: 0.9824
Epoch 465/500 - loss: 0.0630 - acc: 0.9824
Epoch 466/500 - loss: 0.0627 - acc: 0.9828
Epoch 467/500 - loss: 0.0635 - acc: 0.9819
Epoch 468/500 - loss: 0.0626 - acc: 0.9821
Epoch 469/500 - loss: 0.0626 - acc: 0.9821
Epoch 470/500 - loss: 0.0633 - acc: 0.9823
Epoch 471/500 - loss: 0.0633 - acc: 0.9819
Epoch 472/500 - loss: 0.0619 - acc: 0.9827
Epoch 473/500 - loss: 0.0625 - acc: 0.9825
Epoch 474/500 - loss: 0.0626 - acc: 0.9826
Epoch 475/500 - loss: 0.0614 - acc: 0.9825
Epoch 476/500 - loss: 0.0615 - acc: 0.9829
Epoch 477/500 - loss: 0.0611 - acc: 0.9826
Epoch 478/500 - loss: 0.0616 - acc: 0.9825
Epoch 479/500 - loss: 0.0607 - acc: 0.9824
Epoch 480/500 - loss: 0.0614 - acc: 0.9824
Epoch 481/500 - loss: 0.0599 - acc: 0.9833
Epoch 482/500 - loss: 0.0604 - acc: 0.9833
Epoch 483/500 - loss: 0.0611 - acc: 0.9827
Epoch 484/500 - loss: 0.0608 - acc: 0.9833
Epoch 485/500 - loss: 0.0607 - acc: 0.9830
Epoch 486/500 - loss: 0.0602 - acc: 0.9829

```
Epoch 487/500 - loss: 0.0610 - acc: 0.9829
Epoch 488/500 - loss: 0.0600 - acc: 0.9831
Epoch 489/500 - loss: 0.0601 - acc: 0.9827
Epoch 490/500 - loss: 0.0589 - acc: 0.9835
Epoch 491/500 - loss: 0.0584 - acc: 0.9840
Epoch 492/500 - loss: 0.0600 - acc: 0.9832
Epoch 493/500 - loss: 0.0598 - acc: 0.9829
Epoch 494/500 - loss: 0.0590 - acc: 0.9832
Epoch 495/500 - loss: 0.0596 - acc: 0.9835
Epoch 496/500 - loss: 0.0582 - acc: 0.9835
Epoch 497/500 - loss: 0.0583 - acc: 0.9837
Epoch 498/500 - loss: 0.0575 - acc: 0.9837
Epoch 499/500 - loss: 0.0585 - acc: 0.9832
Epoch 500/500 - loss: 0.0575 - acc: 0.9836
```

Checks the model's performance

```
In [ ]: model.eval()

x = torch.tensor(x_test, dtype=torch.float32)
y = torch.tensor(y_test, dtype=torch.long)

with torch.no_grad():
    logits = model(x)
    loss = loss_fn(logits, y)
    acc = (torch.argmax(logits, dim=1) == y).float().mean().item()

print(f"Test loss: {loss.item():.4f}")
print(f"Test accuracy: {acc:.4f}")
```

Test loss: 0.0794

Test accuracy: 0.9748

The model is now trained to almost 98% accuracy. We evaluated the mode on the test set. Can you evaluate it on the training set? Try altering the structure of the network that we set up. Can you change the performance? It will be quite difficult to improve on 98% - but see if you can make it worse! What sort of changes can you make? What effect do they have?

In []:

In []: